Chapter 2

What survives a crash

Suppose a five-node Raft cluster has just elected a leader. The leader proposes a command, and one follower, server 3, appends it to its log and replies yes. The leader counts replies, commits the command, and tells the client it is done. Then server 3 crashes and restarts. On restart it has an empty log and no memory of the term it was in. It immediately starts a new election in the same term it already voted in, votes for a different candidate, and helps overwrite the command the cluster just committed.

The real problem is in the server's memory, not in the protocol. A reboot is not a reset button. A server that wakes up amnesiac is dangerous, because it can rejoin the cluster with a blank slate and break the very guarantees the cluster was relying on. The protocol must decide exactly what state is worth saving and exactly when it is safe to say saved.

> THE CRUX
> *When a server restarts after a crash, what does it have to remember so that it cannot break safety, and how must it write that information so the memory survives?*

A server that wakes up amnesiac

In chapter 1 the crash-recovery fault model said servers may crash and later restart, and that no server lies. The model did not say what a restarting server is allowed to forget. The answer turns out to be: almost nothing that the protocol has already acted on. The moment a server makes a decision that the rest of the cluster can depend on, it has to remember that decision across a reboot.

The easiest way to see why is to watch a single vote disappear. In Raft, a server may grant at most one vote per term. The majority rule guarantees that at most one candidate can win the election for a particular term[1]. But the majority rule only works if the vot-

ers actually remember that they voted. If server 3 grants its vote to candidate A in term 2, then crashes and restarts with no record of that vote, it can grant a second vote to candidate B in the same term. Now term 2 has two winners, the cluster has two leaders, and the split-brain failure from chapter 1 is back.

The same danger applies to log entries. If server 3 appends entry 4, tells the leader yes, and the leader commits the entry on the strength of that yes, then server 3 must still have entry 4 after it reboots. Otherwise the cluster believed an entry was committed because a majority had it, but one of the members of that majority has since lost it. A later leader could overwrite the entry, and safety would be gone.

TRY IT

The `lost-vote` lab shows what happens when a server keeps its term and vote only in memory. It grants a vote, simulates a crash, restarts with an empty `Server`, and grants a second vote in the same term.

GO

```go
type Server struct {
  currentTerm int
  votedFor    string
}

func NewServer() *Server {
  return &Server{}
}
```

OUTPUT

```
term 2: received RequestVote from A
granted vote to A in term 2
CRASH
RESTART with empty memory
term 2: received RequestVote from B
granted vote to B in term 2
SAFETY VIOLATION: voted for both A and B in term 2
exit status 1
```

The output is a safety violation: server 3 voted for both A and B in term 2. The problem is not the voting rule; the problem is that the vote never reached storage.

A rebooted server also has to avoid reintroducing a stale leader. Chapter 1 showed that terms are how Raft fences off an old leader once the cluster has moved on. If a server forgets its term on reboot, it can accept AppendEntries from a leader whose term has already been superseded, or it can start an election in an old term and confuse the cluster. So the current term is part of the memory that must survive.

The opening scenario is therefore three failures in one: the server votes twice, it loses a committed entry, and it forgets which term the cluster is in. All three happen for the same reason: the server treated a process restart as a fresh start.

Volatile versus durable state

A running Raft server holds two kinds of state. **Volatile state** lives in the process and dies with it. **Durable state** is written to **stable storage**, which the Raft paper deliberately leaves abstract: in practice it is usually a local disk, but the protocol only cares that the storage survives crashes[1]. **Persisted state** is the durable state the server writes itself so it can resume correctly.

Volatile state State kept only in memory, such as the commit index and last applied index, which is lost when the process exits and reconstructed on restart.

Durable state State stored on some medium that survives a process crash and reboot.

stable storage Storage that survives crashes, typically a local disk with appropriate sync operations, though the Raft paper deliberately abstracts over the exact mechanism.

Persisted state Durable state that the protocol explicitly writes so it can be reloaded after a restart.

The Raft paper names exactly three persisted fields: `currentTerm`, `votedFor`, and the log entries. `currentTerm` is the latest term the server has seen. `votedFor` is the candidate it voted for in the current term, or null if it has not voted. `log[]` is the sequence of entries, each holding a command and the term in which it was received by the leader[1].

Everything else can be reconstructed. The commit index, the last applied index, the leader's nextIndex and matchIndex tracking, even the server's role as follower, candidate, or leader: all of these are derived from the three persisted fields plus messages from peers. In particular, the commit index needs no persistence because a restarting server can safely reinitialize it to zero[2]. The server will learn the real commit index again as soon as it talks to the leader.

The reason this works is subtle and important. The commit index is a property of the cluster, not a private promise by the server. A server does not need to remember the highest committed entry

because the cluster will tell it again. But the vote and the term are promises. Once server 3 says yes to a candidate, the rest of the cluster can count on that vote having been consumed. Once server 3 accepts a log entry, it has promised not to accept a conflicting one at the same index. Persisting `currentTerm` and `votedFor` prevents a restarted server from voting twice in the same term or replacing log entries from a newer leader with those from a deposed leader[2]. Persisting new log entries before they are counted toward commitment prevents committed entries from being lost or uncommitted when servers restart[2].

TRY IT

The `persisted-vote` lab fixes the lost-vote bug by writing `currentTerm` and `votedFor` to a file and calling `fsync` before replying to RequestVote.

`fsync` A system call that forces all modified data and metadata for a file out to the underlying storage device, establishing an ordering boundary.

GO

```go
func (s *Server) persist() error {
  if err := os.MkdirAll(filepath.Dir(stateFile), 0750); err != nil {
    return err
  }
  b, err := json.Marshal(s.state)
  if err != nil {
    return err
  }
  f, err := os.OpenFile(stateFile, os.O_WRONLY|os.O_CREATE|os.O_TRUNC, 0640)
  if err != nil {
    return err
  }
  if _, err := f.Write(b); err != nil {
    f.Close()
    return err
  }
  if err := f.Sync(); err != nil { // fsync before replying
```

OUTPUT

```
term 2: received RequestVote from A
persisted state: term=2 votedFor=A
granted vote to A in term 2
CRASH
RESTART, loading persisted state
loaded state: term=2 votedFor=A
term 2: received RequestVote from B
refused vote to B in term 2 (already voted for A)
```

> After the crash, the server reloads `term=2 votedFor=A` and refuses the duplicate vote. The fix is small: the server simply has to keep the promise it made.

Note that the basic Raft algorithm does not require the commit index to be persisted[2]. Production systems sometimes do persist more for performance or convenience. etcd's `HardState`, for example, carries term, vote, and commit together[3]. But the core safety argument only needs the three fields.

The write-ahead log

The log is the heart of the design. It is an append-only sequence of records, and that shape is no accident. Appending is simpler and safer than updating state in place because it never overwrites an old record with a new one, and only adds at the end. If a crash happens in the middle of an append, the worst case is a partially written last record, which recovery can detect and ignore. A **write-ahead log (WAL)** has a defining rule: changes to the data files must be written only after the changes have been logged and the log records flushed to permanent storage[4].

write-ahead log (WAL) An append-only sequence of records describing operations that must be applied to the state machine; the source of truth from which the state machine is rebuilt.

This is the same idea that databases have used for decades. PostgreSQL's WAL works the same way: before a page in the main data files is modified, a record describing the change is written to the WAL and flushed to disk. Only the WAL file needs to be flushed to make a transaction durable, rather than every data file the transaction changed[4]. SQLite uses a rollback journal for the same purpose, recording the old state of a page before the page is changed[5], and its atomic-commit procedure assumes that a sector write may be interrupted partway through[5]. All three designs share the same insight: separate the record of what happened from the state that results from it, and make the record the source of truth.

In Raft, the log is the source of truth and the state machine is derived from it. The state machine is the key-value store, the counter, the container scheduler, or whatever the cluster is actually implementing. The log is the ordered list of commands that, when applied in order, produce that state. If two servers have the

same log and apply it in the same order, their state machines match. That is the replicated state machine guarantee from chapter 1 made concrete.

The running command for this chapter is `Set("web-1", "10.0.0.7")`. The leader appends it to its own log as entry 4 in term 2 and sends it to server 3. Server 3 appends it to its log, and only then replies yes. The leader waits for a majority of yes replies, then commits the entry. The command is not really part of the system until it is safely logged on enough servers, and the state machine update comes later.

Because the log is append-only, a server that has accepted entry 4 can never silently replace it with something else. It can only append entry 5, or truncate uncommitted suffixes when a new leader shows it a conflicting branch. That property is what makes the Leader Completeness argument work: the voter that accepted a committed entry still stores it when it later votes for a higher-term leader, because leaders never remove entries and followers only remove entries if they conflict with the leader[2].

fsync and the lie of “written to disk”

A common mistake is to think that calling `write` means the data is safe. It does not. On Linux, `write` usually copies data into the **page cache** and returns immediately. The data may sit in memory for seconds before the kernel flushes it, and even after the kernel sends it toward the disk, the disk may have a volatile **disk write cache** that buffers writes and can lose them if power fails before the cache is flushed.

page cache The kernel’s in-memory cache of file contents; writes normally go here first and are flushed to disk later.

disk write cache Volatile memory on the disk controller or drive itself that can buffer writes and lose them if power fails before the cache is flushed.

Raft therefore relies on `fsync`. `fsync` returns only after all modified in-core data of the file has reached the device, which includes writing through or flushing a disk cache if present[6]. Because it waits, it also gives you an ordering point. A write you issue after `fsync` has returned cannot reach the disk ahead of the writes that `fsync` already flushed.

That ordering is what Raft actually needs. The rule from the Raft paper is that persistent state must be updated on stable storage before responding to RPCs[1]. etcd’s raft library makes the same

contract explicit: no messages may be sent until the latest `HardState` has been persisted to disk, and all entries from any previous Ready batch have been written[3]. The application must write entries first, then `HardState` and snapshot, in that order, or write them together atomically if the storage supports it[3]. The reason for the order is that the message is a promise to the rest of the cluster, and the fsync is the act of keeping that promise.

`fdatasync` is a lighter variant. It is like `fsync` but skips metadata that is not needed to read the data back correctly, though it still flushes metadata changes such as file-size changes because those are needed to read the data back[6]. For an append-only log, `fdatasync` is often enough and can be cheaper.

`fdatasync` Like fsync, but forces only data and a subset of metadata, omitting metadata not required for later reads of the file contents.

There are more caveats. `fsync` on a file does not persist the directory entry that names it, so creating a new log file in the same directory needs a separate `fsync` on the directory itself[6]. And while `fsync` is specified to flush a disk cache if present, some older kernels and lesser-used filesystems do not know how, so the disk cache must be disabled for safe operation[6]. So the honest mental model has two parts. On a correct, modern stack, a returned `fsync` means the device has reported the data as stored. On a stack where some layer cannot flush its cache, that promise quietly fails, and you only find out when the power does.

> MISCONCEPTION
> **“Once fsync returns, nothing can lose the data”**
> On a correct stack, a returned fsync means the device reported the data as stored, cache included. But the guarantee is only as good as every layer below it. Older kernels and some filesystems cannot flush a disk’s cache, and then the cache has to be disabled for fsync to mean what it says[6].

> MISCONCEPTION
> **“As long as I fsync eventually, the order of writes does not matter”**
> Order matters deeply for Raft. A server must persist state before it sends any message that depends on that state, because otherwise a crash can leave the system believing an action happened when the state that justified it was lost.

Back to server 3 and `Set("Web-1", "10.0.0.7")`. The leader sends an AppendEntries RPC containing entry 4 in term 2. Server 3 appends the entry to its log, calls `fsync` on the log file, and only then sends its yes reply. The fsync is the boundary. Before it, the server has not promised anything to the leader. After it, the promise is kept even if the power fails.

Replaying a log safely

When server 3 restarts, it does three things in order. First it reads its persisted term and vote. Then it reads the log from disk. Then it **replays** the log, applying each entry to the state machine, until it reaches the end.

replays The act of reading a log after a crash and reapplying its entries to rebuild state.

The replay rebuilds the in-memory state from the durable source of truth. The commit index and the last applied index were lost in the crash, so the server starts them at zero and walks the log again. For our running example, the log contains entry 4: `Set("Web-1", "10.0.0.7")`. The server applies it and the key-value store contains `web-1 = 10.0.0.7`.

But there is a problem. The server might have already applied entry 4 before the crash. After the crash, replay applies it again. Applying the same command twice must leave the state machine in the same state as applying it once. A command with this property is called **idempotent**. A `set` command is naturally idempotent: setting a key to the same value a second time changes nothing.

idempotent An operation that produces the same result no matter how many times it is applied, so replaying it is safe.

TRY IT

The `wal-replay` lab appends a `Set` command to a WAL, applies it to an in-memory map, simulates a crash, and then replays the WAL to rebuild the map.

```go
func (st *Store) append(e Entry) error {
  if err := os.MkdirAll(filepath.Dir(walFile), 0750); err != nil {
    return err
  }
  f, err := os.OpenFile(walFile, os.O_WRONLY|os.O_CREATE|os.O_APPEND, 0640)
  if err != nil {
    return err
  }
  defer f.Close()
```

```
  b, err := json.Marshal(e)
  if err != nil {
    return err
  }
  if _, err := f.Write(append(b, '\n')); err != nil {
    return err
  }
  if err := f.Sync(); err != nil { // fsync the WAL before applying
    return err
  }
  fmt.Printf("WAL entry: %s\n", string(b))
```

OUTPUT

```
WAL entry: {"op":"set","key":"web-1","value":"10.0.0.7"}
applied: web-1 -> 10.0.0.7
state before crash: map[web-1:10.0.0.7]
CRASH (lost in-memory map)
RESTART
replaying WAL
replay: set web-1 10.0.0.7
applied: web-1 -> 10.0.0.7
state after replay: map[web-1:10.0.0.7]
```

The map is lost in the crash, but the WAL survives. Replaying the entry applies the same `Set` again, and the final map matches the state before the crash.

Not every operation is idempotent by nature. A command like `Add(10)` would double-count if replayed. Databases handle this with techniques such as per-page log-sequence numbers (LSN).

The Algorithms for Recovery and Isolation Exploiting Semantics (ARIES) recovery algorithm makes redo idempotent by consulting the LSN stored on each page to decide whether a logged operation still needs to be redone[7]. PostgreSQL periodically writes a full image of a page into the WAL before modifying that page on disk, so a partially written page can be restored from the WAL during recovery, and every WAL record carries a 32-bit Cyclic Redundancy Check (CRC-32C) checksum so an incomplete record can be detected instead of replayed as if it were complete[8].

For Raft, the simplest approach is to make state-machine commands idempotent. A key-value `set` is one example. Counters and inventory systems need more care. The replay rule also implies

that a persistent state machine, one that has already applied most entries before a restart, must persist its last applied index to avoid reapplying everything[2]. That is why real systems often keep a snapshot or a checkpoint: they replay only the log entries after the snapshot, not the entire log from the beginning of time.

The full picture, then, is this. A Raft server keeps three pieces of state on disk: its term, its vote, and its log. Before it sends any message that depends on those pieces, it fsyncs them. On restart it reloads them and replays the log to rebuild the state machine. The state that matters is the state the server has already promised to the cluster, and everything else can be derived again.

Check yourself

1. Server 3 crashes after appending entry 4 and replying yes to the leader, but before applying the entry to its state machine. After restart, what does it do, and why is the result correct?
2. Why is `commitIndex` safe to lose on a restart, while `votedFor` is not?
3. A server receives a RequestVote in term 2 and is about to grant it. What must happen before it sends the grant reply?
4. Give an example of a non-idempotent state-machine command and explain why replaying it after a crash would be unsafe.
5. etcd recommends running an odd number of members and at least three. How does that recommendation connect to the durable-state ideas in this chapter?

Answers are at the end of the chapter.

Answers

1. On restart the server reads its persisted log, which still contains entry 4. It replays the log from the beginning, applying entry 4 to the state machine. The result is correct because the log entry was persisted before the yes reply, and because `Set` is idempotent, so applying it again after a crash does no harm.
2. `commitIndex` is a property the server learns from the cluster, and it can be reinitialized to zero so the leader will update it.

`votedFor` is a promise the server made, and if it is lost, the server can vote twice in the same term and break election safety.

3. The server must persist `currentTerm` and `votedFor` to stable storage and fsync before sending the reply. The reply is a promise that depends on the persisted state.
4. `Add(10)` to a counter is not idempotent. If the server applied it before the crash, then replay applies it again, the counter ends up 20 higher instead of 10. A real system would need a last-applied index, per-record sequence numbers, or an idempotent encoding such as `Set(balance, 110)`.
5. An odd-sized cluster has a majority of `floor(n/2)+1`, and adding one node to an odd-sized cluster always increases the quorum, so there is no reason to add the extra node just for fault tolerance[9]. Three members tolerate one failure and are the minimum recommended size, because removing a member from a two-member cluster can leave it unable to make progress[3]. Durability matters because the cluster can only survive crashes if a majority of members keep their promises across reboots.

Sources

[1] In Search of an Understandable Consensus Algorithm (Extended Version), Extended version published 2014-05-20 (USENIX ATC '14 paper plus extra material)
https://raft.github.io/raft.pdf

[2] Consensus: Bridging Theory and Practice (PhD dissertation), Stanford University, August 2014; dissertation source commit 0c10ea1b (2016-05-24)
https://raw.githubusercontent.com/ongardie/dissertation/0c10ea1b53d73bf6d163bdde8e4141a1cbd8f36a/online.pdf

[3] etcd-io/raft (Go Raft library), tag v3.7.0 (commit b867cf13f6bc0dae21204302df97bc2355c3af55)
https://github.com/etcd-io/raft/tree/v3.7.0

[4] PostgreSQL 17 Documentation, 28.3. Write-Ahead Logging (WAL), PostgreSQL 17.11 documentation (page dated 2026-08-13)
https://www.postgresql.org/docs/17/wal-intro.html

[5] Atomic Commit In SQLite, sqlite.org/atomiccommit.html, last updated 2026-04-21
https://sqlite.org/atomiccommit.html

[6] fsync(2) - synchronize a file's in-core state with storage device, Linux man-pages 6.19-1 (2026-02-08); read locally as 'man 2 fsync'
https://man7.org/linux/man-pages/man2/fsync.2.html

[7] ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging, ACM Transactions on Database Systems 17(1):94-162, March 1992 (PDF read via people.eecs.berkeley.edu/~brewer/cs262/Aries.pdf)
https://people.eecs.berkeley.edu/~brewer/cs262/Aries.pdf

[8] PostgreSQL 17 Documentation, 28.1. Reliability, PostgreSQL 17.11 documentation (page dated 2026-08-13)
https://www.postgresql.org/docs/17/wal-reliability.html

[9] etcd documentation: FAQ (quorum, membership, failure tolerance), etcd v3.5 docs, page last modified 2024-12-05
https://etcd.io/docs/v3.5/faq/